

Learning When to Recompute for Efficient Video Object Detection

Wayne Hall

Department of Computer Science
Florida Polytechnic University
Lakeland, FL, USA
whall3063@floridapoly.edu

Shriraj Mandulapalli

Department of Computer Science
Florida Polytechnic University
Lakeland, FL, USA
smandulapalli3528@floridapoly.edu

Abstract—Video object detection is often accurate but heavy; having a strong object detector run and recompute each frame in a video is often unnecessary when objects change slowly or barely move, yet generic skipping risks inaccurate bounding boxes after movement, occlusion, or a shift in appearance. This paper studies the results of using a learned gate to determine whether to invoke a full detector again versus reusing prior computed predictions. We utilize the UA-DETRAC multi-object detection dataset and YOLOv8m to form temporally offset pairs and quantify bounding box stability using Hungarian-matched Intersection-over-Union (IoU) between detections [2]. We then calculate a threshold on mean matched IoU to determine recompute versus keep under a video-level train/validation/test split. Each pair is encoded as a two-channel tensor, including Farneback optical-flow magnitude and Gaussian-blurred absolute difference. We then create a lightweight convolutional neural network (CNN) to map these cues to the recompute/keep decision. (then insert results where when done).

Index Terms—video object detection, optical flow, gaussian blur, convolutional neural network, intersection over union

I. INTRODUCTION

Video object detection is rising in usage quickly; from vehicle detection at traffic lights to continuous streaming with autonomous vehicles, more and more companies are utilizing the same method for object detection. These detectors are running on long frame streams where accuracy and throughput are crucial, so it's important to ensure that the computation is as effective as possible.

Current strong detectors (YOLO family models, CNNs, etc) already do a good job at providing bounding boxes for a variety of objects, but the cost grows roughly linearly with frames when running at every timestep. This becomes problematic for long recordings, real-time pipelines, or even edge devices where cost and optimization become much more important than just results.

Uniform subsampling provides an easy approach but becomes misaligned with the scene dynamics. Sometimes nothing will change for many frames and other times it'll have spontaneous motion, occlusion, or appearance changes. If there is no rule to refresh detections, then it opens up the possibility of risking stale or misaligned bounding boxes. We aim to

create and apply an adaptive policy, not just "select fewer frames".

We treat the process of recomputing bounding boxes as a learned binary decision. This allows us to invoke the full detector again or reuse prior predictions by keeping the previous boxes for the current frame. Utilizing optical flow magnitude, blurred absolute frame difference, and the temporal gap between the frames ends up being cheaper in comparison to a full detection process [3].

We end up using the UA-DETRAC (traffic, video, and multi-object) dataset and use YOLOv8m to produce per-frame boxes. Pair-level labels are derived and stored in a tabular manifest (parquet). We create train/validation/test splits while respecting video identity and preventing leakage.

Using this, we run YOLOv8m on each extracted frame to obtain bounding box detections and then we compute temporally offset pairs. For each pair, we match the boxes between frames with a one-to-one assignment found by the Hungarian algorithm using pairwise IoU [6]. Then we summarize how stable the boxes are and turn that into recompute versus keep labels with a fixed IoU threshold. Lastly, we train a small CNN on the optical-flow magnitude, Gaussian-blurred absolute difference, and the frame gap to predict whether the detector should run again or reuse the previous frame's box.

II. PROPOSED METHOD

A. Overview

Current and modern single-frame object detectors can create and deliver strong bounding boxes on multiple images, but applying them uniformly at the native frame rate of traffic videos will end up inflating the computation, latency, and power. When nothing is occurring in the scene (e.g it's static), many consecutive frames support the same object locations. If motion, occlusion, or sudden appearance changes occur, it would have to rely on a stale bounding box which can be risky. Therefore, we study and create a binary decision: at a given pair of frames separated by a gap but from the same video, should the algorithm run the heavy detector again (recompute) or is it sufficient to reuse the previous prediction (keep). This is the question we aim to figure out.

The training signal for the gate does not manual labeling. We instead create proxy labels from how much agreement

there is between YOLO detections on the two frames after matching bounding boxes so that they're only used at most once. In reference to IoU, high agreement supports keeping the previous bounding box while low agreement supports the recomputation of the bounding box. At inference time, the gate model only sees low-cost maps created from the pixels and the gap difference. This makes the learned rule only connect to appearance and motion, not bounding box tensors.

B. Data & Detector Output

Our system is initially tested on the UA-DETRAC dataset which contains multiple videos of a traffic light perspective with cars driving on the road. We end up taking the videos, converting them into separate frames, and giving them specific names which is useful throughout testing. In the initial stage of the system, we use and load ultralytics YOLO weights to predict on all the frames/images [1]. It only iterates over chunks at a time to prevent memory limitations on long clips. The results are filtered before writing labels for each image. We only utilize classes containing a car, motorcycle, bus, or truck. All other classes and any detection with a confidence below 0.25 are dropped.

After that, each detection becomes one line in a YOLO-compatible text file containing the normalized center coordinates, normalized width and height, and confidence. Every processed frame has its image properties logged into a manifest, including the height, width, path, frame index, and detection count. This stage creates consistent pseudo-labels on all frames the later stages utilize.

C. Pair Creation & Box Association

Stage two ingests the frames grouped by video and sorted by integer frame number. For each anchor frames t_a at position i and the temporal offset Δ drawn from our configuration file, we attempt to create a pair (t_a, t_b) with

$$t_b = t_a + \Delta \quad (1)$$

whenever the index of $i + \Delta$ lies within the sequence. If it does not reside within the sequence, then we scrap the pair. The offsets are not coded in the system and is chosen by developer preference. We decided to select utilizing prime numbers since it naturally prevents k frame pattern matching.

Each valid pair will have the detections parsed. An empty label file will yield an empty 0×6 array and a single-line file is then reshaped to one row. Each detection row stores the class and box parameters. The IoU between two detections uses the normalized center parameters $(x_c, y_c, w, h) \in [0, 1]^4$ while the corners are reconstructed and the axis-aligned intersection area and union are computed in a continuous space of $[0, 1]^2$.

Let $\mathcal{B}_x$ and $\mathcal{B}_y$ denote the (possibly empty) sets of boxes in frames x and y . We build the IoU matrix $M \in \mathbb{R}^{|\mathcal{B}_x| \times |\mathcal{B}_y|}$ with entries $M_{pq} = \text{IoU}(b_p^{(x)}, b_q^{(y)})$. The boxes are matched *one-to-one* across the two frames (each box used at most once). The Hungarian algorithm selects the matching that maximizes the sum of pairwise IoUs over all the matched pairs, which is the standard optimal assignment on a bipartite graph weighted by

IoU. If the IoU of a matched pair doesn't satisfy $M_{pq} > 10^{-6}$, then it will be discarded.

If either frame has zero detections, we match short-circuits to $n_{\text{matched}} = 0$ and $\overline{\text{IoU}} = 0$. Every pair row (video identifier, t_a , t_b , Δ , and detection counts) are written to our parquet file. We have checkpoint files created every N pairs so interrupted runs can resume by skipping (video, t_a , Δ) triples that are already present.

D. Label Generation & Dataset Splits

Moving forward, the next stage will read the parquet and drop every row which has number detections set to 0. Those pairs don't provide a meaningful stability comparison between non-empty box sets.

The labels come from $\overline{\text{IoU}}$ vs a scalar τ . We decide to recompute if $\overline{\text{IoU}} < \tau$ or keep if otherwise. The choice of τ controls how often CNN will decide to recompute vs keep. That balance depends on Δ because larger gaps usually lower $\overline{\text{IoU}}$. We calculated multiple τ values on the pair table (Table 1). We found that $\tau=0.50$ is too lenient since it'll almost always select *keep* at small Δ and it has weak supervision for *recompute*. $\tau \in 0.90, 0.95$ is too strict so it'll rarely select *keep*, especially at large Δ . We select and adopt $\tau=0.70$ since it retains a non-trivial *keep* rate while still assigning enough *recompute* examples at larger Δ for the model to learn when motion or instability matters.

TABLE I
FRACTION OF FRAME PAIRS ASSIGNED LABEL *keep* ($y=1, \overline{\text{IoU}} \geq \tau$), BY TEMPORAL OFFSET Δ , FOR POTENTIAL THRESHOLDS τ .

$\tau \setminus \Delta$	1	3	5	7	11	13
0.50	0.999	0.878	0.768	0.707	0.627	0.594
0.70	0.969	0.703	0.555	0.408	0.273	0.244
0.80	0.840	0.496	0.272	0.179	0.114	0.106
0.90	0.497	0.109	0.055	0.037	0.029	0.031
0.95	0.085	0.019	0.013	0.010	0.009	0.010

Next, the train / validation / test splits are grouped by videos and then broadcast back to every pair row. No pair from one split can appear in another, allowing for the sets to remain truly independent of the training process. The video group that only appears once are detailed as singles. We stratify by video group so each split still samples every common group fairly. The stratification uses multiples first. When possible, it'll target 60% train and 40% holdout on videos. The holdout is then split again 50/50 into validation versus test. If there are too-few groups, it will fall back to random splits with a random state. Rare holdout groups and other singletons are placed into train.

E. Gate Input Representation

During the training process, each example is one row from the labeled pair table. It contains a video clip id, the two frames t_a and t_b , the offset Δ and the binary label. To turn the pair into something readable by the gate without running a YOLO model again, we read two still images from our pipeline and optionally resize them if downsampling is enabled.

From the first frame to the second frame, we calculate the dense optical flow with the Farneback algorithm and use the magnitude [4]. In parallel, we compute a Gaussian blur and take the absolute difference of both images which highlights the appearance change while ignoring some noise. Both maps are scaled into comparable ranges before stacking.

Although Δ is known when we build the row, our training only involves providing the two-channel tensor x and the label y . The offset is not provided as a numeric input and is strictly used for determining which two frames were paired and how x looks for that pair.

Minibatches are formed with shuffling the training split. No shuffling occurs for the validation and testing splits. The training dataloader drops the last incomplete mini batch so every step guaranteed sees a full batch. If a GPU is available to speed this up, our code will factor that in. The configuration file contains batch size, image size, blur settings, flow scaling, and some data paths.

F. Gate Network & Training

Our gate is a small convolutional neural network (CNN) that maps the two-channel x to a single logit z for each pair [5]. We do not utilize a final sigmoid since the loss expects logits.

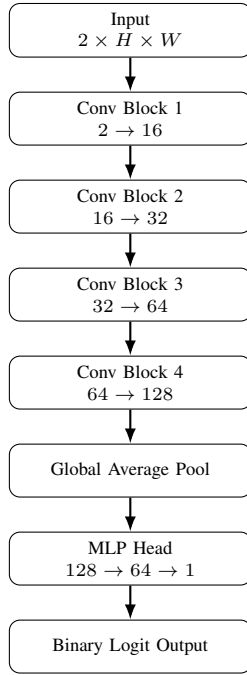


Fig. 1. Overview of our CNN architecture. Each convolutional block contains convolution, batch normalization, ReLU activation, and max pooling.

Figure 1 outlines the layout of our CNN. After the input, we have four stacked 3×3 convolution stages where batch normalization and ReLU occur allowing the representation to change from 2 channels to 128. The next stage computes the global average pooling, creating a 128-dimensional vector. Lastly, a compact head will apply dropout and map the 128

to 64 with ReLU and then maps 64 to 1 to finally produce the logit z .

The training stage will treat this a binary classification using logits. The objective is binary cross-entropy with logits, coupling the cross-entropy and standard sigmoid in a numerical way. We use the Adam optimization and a learning rate of 10^{-3} . We have a stepwise learning rate which reduces every five epochs, allowing the last phase of training to stabilize faster.

We train for exactly 15 epochs. After each step, we measure the average loss on the training and validation sets. The best weights are chosen by lowest validation loss, not the highest accuracy. We save these weights for future snapshots of testing.

III. EXPERIMENTAL SETUP

A. Evaluation Scope

Our main benchmark measures whether the CNN agrees with the automatic recompute/keep labels on data from the UA-DETRAC dataset. After training, we report the performance on the test defined by video-level splits. The labels on the rows are automatic since they come from the YOLO outputs and Hungarian matching, not human annotation. Due to this, our reported accuracy means agreement with the stability rule, not agreement with the independent ground truth for when recomputation is "correct".

B. Dataset Scale

We summarize the artifacts produced before the training occurs, sampling 100 unique videos with valid frames. For transparency, we summarize scale in terms of how many unique videos contribute frames and how many frame pairs survive the filters, since those will define our supervision. We remove pairs where at least one frame has no vehicle detections since our stability will not be meaningful if one side of the pair is empty.

We report these counts broken out by splits to showcase whether the evaluation is balanced across recompute and keep decisions, whether large temporal gaps dominate the table, and whether the test portion is large enough to provide sufficient results. Table I explains why we utilize the IoU threshold $\tau = 0.7$, since it provides the best data split.

C. Configuration Protocol

All of our results presented use a fixed configuration from end to end. Detection is produced the same way for every frame kept in UA-DETRAC: one YOLO family detector, the same vehicle detection filter, same score cutoff for dropping boxes with a low confidence, and a label file layout.

Pairing is also fixed: we anchor frames in temporal order and pair them only at the temporal offsets Δ that we selected before looking at performance. Supervision for the gate is entirely automatic since the boxes are Hungarian-matched in IoU, summarized by the mean matched IoU and thresholded with τ .

The gate in the pipeline is also frozen with an optional parameter of downsampling for resizing the images or not. We implemented the ability to resize the frames / images but did not go through with it to keep the computation as similar as possible. Farneback flow magnitude, Gaussian blurred absolute difference, and the normalizations that put both channels on a binary comparable scale are always fixed. Training follows a single logit based binary objective with a standard adaptive optimizer, a fixed set of epochs, a learning rate with a step scheduler, and a checkpoint mechanism.

D. Metrics

The primary metric we score is how often the gate matches the automatic decision of recompute / keep labels. Predictions use the logit z with the $z > 0$ rule used during training. The reference y comes only from the YOLO detections, Hungarian matching, and the IoU threshold τ , so the accuracy is agreement with the stability policy. We also look at the average latency and mean IoU across our testing data, including videos with bounding boxes computed by YOLOv8m, k -skips when $k = 10$, and our learned gate. The average latency and mean IoU of the YOLO model is used as a baseline to determine the improvement in computation when utilizing our learned gate system.

IV. RESULTS & ANALYSIS

A. Validation and Testing Results

We evaluate the checkpoint with the lowest validation loss during training on our validation and test splits. Instead of collapsing everything into one overall percentage, we break the accuracy by temporal gap Δ since the pairing policy feeds the same CNN all the gaps but doesn't provide Δ as a specific input. This allows different patterns across Δ to be meaningful about what motion and appearance cues are capturing. Table II showcases the testing split; the accuracy is at its highest when at $\Delta = 1$, then declines through $\Delta = 7$. After, it slightly recovers in accuracy when using large gaps.

Table III shows the same layout on the validation split and shows similar results. At $\Delta = 1$, it is nearly perfect since it's much easier to select *keep* as a decision. Moving towards $\Delta = 7$, a similar dip occurs since it's harder to determine to recompute or keep with that gap. However it becomes slightly easier as the gap grows since it can confidently select *recompute* due to the difference between the frames.

The U-shaped trend in our accuracy across varying gaps aligns well with what we would expect. Small and large gaps are kept and skipped, respectively, with high confidence, while intermediate gaps perform well above chance. This tells us the model is able to generalize well, giving proper predictions.

B. Latency Results

Outside of label agreement, we also care whether the addition of the gate reduces the cost relative to always running the heavier detector. Table IV shows the results of the average latency and mean IoU for the entire test set when using the original YOLOv8m, our learned gate, or a k -skips (when

TABLE II
VALIDATION ACCURACY RESULTS ACROSS TEMPORAL GAPS Δ .

Gaps Δ	Correct	Total	Accuracy
1	24,991	26,559	94.1%
3	23,957	26,518	90.3%
5	23,147	26,476	87.4%
7	22,552	26,435	85.3%
11	22,738	26,355	86.3%
13	22,965	26,315	87.3%

TABLE III
TESTING ACCURACY RESULTS ACROSS TEMPORAL GAPS Δ .

Gaps Δ	Correct	Total	Accuracy
1	29,949	30,230	99.07%
3	26,949	30,190	89.27%
5	23,867	30,150	79.16%
7	23,956	30,110	79.56%
11	24,697	30,030	82.24%
13	24,926	29,990	83.11%

$k = 10$) algorithm. The average latency for the YOLO model is much higher since it repeats that computation every time. The mean IoU is 1 since it is our baseline. The important part of this table is the comparison between our learned skip versus k -skips. With k -skips, the latency is significantly cheaper but pays for it in quality, dropping to 0.723 mean IoU. With our learned skip method from the gate, we can see the best of both worlds as it performs quicker than the Full YOLO model and also has a great score in the mean IoU. Our method is about 2.2x faster than the Full YOLO but still slower than the k -skip method. Overall, using the data from the table, we showcase that utilizing a data-driven learned gate can improve computation without sacrificing that much performance.

TABLE IV
LATENCY AND IOU COMPARISON ACROSS INFERENCE METHODS.

Method	Avg Latency (ms)	Mean IoU
Full YOLO	19.94	1
k -Skips	1.92	0.723
Learned Skip	8.98	0.831

C. Convolution Neural Network Validation Metrics

Lastly, we also summarize the gate with both threshold-free and threshold-based classification scores on the validation set and testing set. We calculate area under the curve (AUC) of the receiver operating characteristic (ROC) to showcase how likely it is to start encountering false-positives.

TABLE V
VALIDATION AND TEST SET PERFORMANCE METRICS.

Metric	Test	Validation
AUC	0.9446	0.9640
Precision	0.8385	0.8313
Recall	0.8907	0.9271
F1 Score	0.8639	0.8766

As seen in Table V, the AUC value being near 1 in both test and validation splits shows that regardless of what threshold we use for triggering the recompute decision, we catch a good proportion of true positives while minimizing false positives. The precision, recall, and F1 score shows that the positive predictions are mostly right and we retrieve most of the true positives under our reported positive-class definition.

D. Analysis

Together, these tables assist in showcasing the possibilities of using a learned gate to determine recompute or keep for bounding boxes. The gate performs best when Δ is small or large since the flow and blurred difference provide a clear signal for determining the proper outcome. Decisions become harder in the middle ground since differences aren't as obvious, opening up the potential for future work and improvement. At a system level, our gate showcases improved computation versus the baseline YOLO model while also providing better mean IoU than other potential substitutions such as k -skips.

V. DISCUSSIONS & LIMITATIONS

Our experiments have demonstrated that by using a learned gating mechanism, calls to large object detection models can be minimized while maintaining performance. By feeding our model frame-to-frame differences, specifically flow magnitude and absolute difference, it can infer future motion, aiding in determining when bounding boxes need to be recomputed. This can be seen in the per-gap breakdown, showing it is not simply learning shortcuts but rather meaningful understandings of when to skip.

A. Limitations

Since we use YOLO predictions as the ground truths when YOLO is wrong our model may potentially learn to predict these failures and skip rather than focusing on the motion of objects in the scene. Since our focus for this project is vehicles the model ignores motion from humans and other objects.

B. Future Work

Some potential expansions of this could extend the training data to different domains like humans, planes, boats, or trains. Computing average motion in videos could allow for a better distribution of motion between training test and validation improving generalization across motion levels.

VI. CONCLUSION

Throughout our research and development phase, we studied when to re-run a heavy YOLO detector versus reusing prior existing bounding boxes for efficient video object detection. Our pipeline builds automatic supervision from temporal frame pairs and Hungarian IoU matching summarizes how stable detections are across an offset gap. Our gate is a compact CNN with two channels for optical flow and Gaussian blur. It is trained with logit binary cross-entropy and a checkpoint that is auto-saved by lowest validation loss. On pairwise classification, agreement with the automatic rule is highest

with a small Δ and solid with a large Δ . It becomes harder in the mid-range due to the decision making becoming less obvious. In terms of runtime, we tell the same story: full per-frame YOLO has the most latency and creates the baseline mean IoU. k -skips is the fastest but reduces quality. The learned gate is faster than the YOLO model and provides better quality than the k -skip method, which makes it a suitable replacement when efficiency needs to be maximized. In the future, this learned gate can be tested on other class objects, further extending the viability of this idea.

ACKNOWLEDGMENT

Support for this paper was provided by the CAP4410 (Computer Vision) course instructor, Dr. Muhammad Abid.

The support of these organizations is gratefully appreciated. The opinions, findings, conclusions or recommendations expressed in this publication are those of the authors and do not necessarily reflect the views of the study sponsor.

REFERENCES

- [1] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," GitHub repository, 2023. <https://github.com/ultralytics/ultralytics>.
- [2] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," arXiv preprint arXiv:1506.02640, 2015.
- [3] N. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," arXiv preprint arXiv:2105.07901, 2021.
- [4] P. Sand and S. Teller, "Particle Video: Long-Range Motion Estimation Using Point Trajectories," Massachusetts Institute of Technology, Cambridge, MA, USA, Tech. Rep., 2008. <https://www.diva-portal.org/smash/get/diva2%3A273847/FULLTEXT01.pdf>
- [5] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," Proc. IEEE, vol. 86, no. 11, pp. 2278–2324, Nov. 1998. https://www.researchgate.net/publication/2985446_Gradient-Based_Learning_Applied_to_Document_Recognition
- [6] H. W. Kuhn, "The Hungarian Method for the Assignment Problem," Naval Research Logistics Quarterly, vol. 2, no. 1–2, pp. 83–97, 1955. <https://web.eecs.umich.edu/~pettie/matching/Kuhn-hungarian-assignment.pdf>